

Prime Attention for Multivariate Time Series Forecasting (Simplified Reproduction)

Model Research Report (Sample spec, PyTorch) — model_type: TimeSeries — based on Lee & Clark, *Dynamic Relational Priming Improves Transformer in Multivariate Time Series* (arXiv:2509.12196)

1. Overview and model card

Standard attention presents the same static token representation in every pair-wise interaction. Prime attention instead modulates each key/value vector with a pair-specific 'primer' F_{ij} derived from the lead-lag cross-correlation structure between channels, so that different channel pairs interact through tailored representations, at the same asymptotic cost. The paper reports up to 6.5% forecasting gains on MTS benchmarks. Model card:

```
model_name: PrimeAttentionModel
model_type: TimeSeries
description: Channel-attention forecaster whose keys/values are modulated by pairwise cross-correlation primers; predicts the next-step value of the target channel (channel 0).
hyperparameters: d_model = 16; n_lags = 8
training_hyperparameters: n_epochs = 20; lr = 1e-3; early_stop = 5; batch_size = 128; weight_decay = 1e-4
```

2. Integration contract (mandatory)

The deliverable is ONE Python file named model.py containing exactly one class subclassing torch.nn.Module and the module-level assignment model_cls = <YourClass>. The automated runner applies this contract verbatim:

```
import torch
from model import model_cls
m = model_cls(num_features=10, num_timesteps=4)
for _, param in m.named_parameters():
    param.data.fill_(1.0)
out = m(torch.full((32, 4, 10), 1.0))
assert out.shape == (32, 1) and torch.isfinite(out).all()
```

Consequences: (a) `__init__(self, num_features, num_timesteps, **kwargs)` with defaults for everything else; (b) forward receives shape (batch_size, num_timesteps, num_features) and returns (batch_size, 1) - do NOT permute the input, timesteps are already axis 1; (c) with every nn.Parameter overwritten by 1.0 and a constant all-ones input the output must stay finite: no division by parameters, no BatchNorm (zero-variance batch gives NaN), prefer LayerNorm with eps or tanh/sigmoid gates; (d) fixed constants belong in register_buffer (buffers are NOT overwritten); (e) no side effects at import time, no try-except, no top-level main; the demo of this report must live inside an `if __name__ == '__main__':` guard. Only torch and the Python standard library may be imported. model_type: TimeSeries.

3. Model formulation

forward(x) with x of shape (B, T, C), where C = num_features is the number of channels and T = num_timesteps the look-back. Target channel is channel 0; output (B, 1) is the predicted next-step value of channel 0.

Embedding: per channel c, $z_c = \tanh(x[:, :, c] @ W_e^T + b_e)$ with W_e a nn.Linear(num_timesteps, d_model) shared across channels $\rightarrow$ z of shape (B, C, d).

Standard attention: $q_i = z_i W_q$, $k_j = z_j W_k$, $v_j = z_j W_v$ (nn.Linear(d_model, d_model)); $a_{ij} = \text{softmax}_j(q_i \cdot k_j / \sqrt{d})$; $o_i = \sum_j a_{ij} v_j$.

Primer: lag features R_{ij} in R^L ($L = \min(n_lags, \text{num_timesteps})$) from the circular cross-correlation of the RAW series x_i and x_j via torch.fft ($R = \text{IFFT}(\text{FFT}(x_j) * \text{conj}(\text{FFT}(x_i)))$, first L lags, tanh-compressed). $F_{ij} = 1 + \tanh(R_{ij} @ W_p^T)$, W_p a nn.Linear(L, d_model) (the primer projection). Modulated $k'_j = k_j * F_{ij}$, $v'_j = v_j * F_{ij}$ element-wise, and $o'_i = \sum_j \text{softmax}_j(q_i \cdot k'_j / \sqrt{d}) v'_j$.

Readout: $y = \text{Linear}(d_model, 1)(o'_0)$, returned as (B, 1). Constructor keyword prime: bool = True switches between prime attention and the standard-attention ablation; attention: bool = True disables attention entirely ($o_i = z_i$, the

channel-independent baseline). Both flags are constructor kwargs with defaults, so the acceptance test's `model_cls(num_features=10, num_timesteps=4)` call stays valid.

4. Stability requirements

With all parameters filled with 1.0 and a constant all-ones input: tanh embeddings keep z bounded, softmax is normalized, $F_{ij} = 1 + \tanh(\cdot)$ stays in $[0, 2]$; output finite. Use $\text{eps} = 1\text{e-}6$ anywhere a division occurs. No BatchNorm. FFT of a constant series is finite; guard the lag truncation for small num_timesteps ($L = \min(8, T)$).

5. Demo data and evaluation (`__main__` guard only)

Inside the guard, generate a synthetic dataset with `torch.manual_seed(42)`: $C = 8$ channels, $T_{\text{total}} = 2000$ steps. Channel 0 = sine + AR(1) driver; channel 1 = channel 0 delayed 3 steps + noise (lead-lag pair); channel 2 = $0.8 * \text{channel 0}$ + noise (instantaneous); channel 3 = independent AR(1); channels 4/5 = lead-lag pair with lag 5; channels 6/7 = independent seasonal + noise. Slide windows with look-back 24 (construct the model with $\text{num_timesteps} = 24$, $\text{num_features} = 8$) predicting the next value of channel 0. Chronological 70/15/15 split. Train with Adam (lr $1\text{e-}3$, weight_decay $1\text{e-}4$, batch 128, max 20 epochs, early stop patience 5 on validation MSE). Report test MSE for three variants of the same class: (a) attention = False baseline, (b) prime = False, (c) prime = True, plus the relative improvement of (c) over (b). Expect prime attention to match or beat standard attention, with the clearest gains on lead-lag structure.

6. Reference

Hunjae Lee, Corey Clark. Dynamic Relational Priming Improves Transformer in Multivariate Time Series. arXiv:2509.12196, 2025. Official code: <https://github.com/timlee0131/Prime-Attention>